

厦門大學

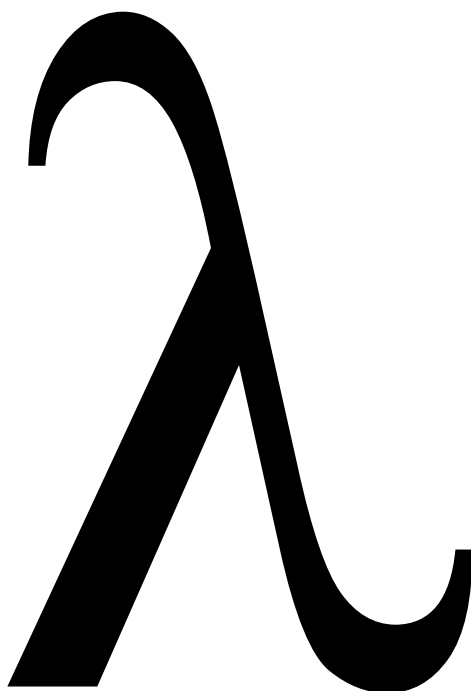
非经典计算
课程论文

**λ演算与函数式编程：
历史、概念和为什么**

姓名	陈海健
学号	22920172204071
学院	信息学院
专业	智能科学与技术
年级	2017级

二〇二零年五月

λ演算与函数式编程：历史、概念和为什么



本文是以一个编程语言爱好者和λ-演算、函数式编程的初学者的角度，以科普的风格所写。作者希望能从λ演算和函数式编程语言的早期历史出发，梳理二者的基础概念，分析常见函数式语言的特性，并给出个人的看法和观点。

陈海健 @ 2019

修订日志：

初稿 2020.4 主体内容

修订 2020.5 文档重排版；补充Q&A；λ演算运算规则部分重写；补充SKI组合子演算和ADT的解释；补充FP的缺陷讨论；表述改进和错误订正

目录

- **λ演算与函数式编程：历史、概念和为什么**
 - 1920s 组合子逻辑的诞生
 - 组合子逻辑的早期历史
 - SKI组合子演算
 - 1930s λ演算的诞生
 - λ-演算的早期历史
 - λ-演算的基本概念
 - 定义
 - 柯里化
 - 自由变量和绑定变量
 - 1940s-1950s的发展
 - 30年代后的一些历史
 - λ演算的变换规则
 - 1960s 编程语言
 - John McCarthy与Lisp
 - Peter Landin与Algol 60
 - Corrado Böhm的贡献
 - 早期的函数式编程语言
 - 之后的历史
 - 1970s & 2010+ 函数式编程语言
 - 函数式编程的概念
 - Lisp的独特魅力
 - 方言
 - S-表达式
 - 高阶函数
 - 宏
 - ML的类型系统
 - 代数数据类型
 - 参数多态和类型推导
 - 模式匹配
 - Miranda / Haskell 的惰性计算和纯函数
 - 惰性计算
 - 纯函数
 - 为什么我们需要FP?
 - Q&A
 - 参考资料

1920s 组合子逻辑的诞生

组合子逻辑的早期历史

在Lambda演算发明之前，更早发明、更加简单、同样强大的是**组合子逻辑**，且二者的历史是相互关联的。组合子逻辑是一种符号系统，最初由Moses Ilyich Schönfinkel发明，后由Haskell Curry重新发明。

Schönfinkel在他的1924年论文后再也没有发表相关的论文，据称他患上了心理疾病，长期在疗养院中度过[Curry, 1927, p.3], [Kline, 1951, p.47]。后来对组合子逻辑的贡献大都是Curry做出的。我们一般将组合子逻辑的发明归功于这两个人。

Schönfinkel的主要目标是**消除约束变元**。他认为，约束变元只不过是一种任意的语法元素。为此，他引入的**组合子**的概念（一种特殊的函数），加上函数应用，就构成了整个符号系统。他引入了B, C, I, K, S五个组合子，并证明了，只用K和S足以定义其它三个组合子，而且，使用S, K加上一个等价于NAND的逻辑运算符，这个系统可以在不需要约束变元的条件下产生所有谓词逻辑表达式。

同时期，Schönfinkel还指出，多变量的函数应用可以用单变量函数应用替代。

$$F(x, y) \Rightarrow (f(x))(y)$$

这就是被现代计算机科学所称作的“**Curry化**”，由于Haskell Curry在其论文中的使用而得名，尽管Curry一再声明这是Schönfinkel的发明。

Curry在研究命题逻辑期间，发现代入(Substitution)规则比其它规约规则要复杂得多，而且那时对代入操作的研究甚少。

Russell和Whitehead著名的《数学原理》对此根本没提，Hilbert和Ackermann编写的一本教科书的第一版，对这个规则的叙述居然还是错的。

在1926-27年间，Curry开始寻求一种将代入过程分成更简单的步骤的方法。在此指引下，他几乎将Schönfinkel的组合子概念重新发明了一遍。1930年，他建立了一个组合子构成的形式系统，包含了完备性证明以及一个抽象算法。这个算

法虽然复杂，但是比多数后来者都来得高效。在20世纪70年代人们试图将组合子引入编程语言的时候，显得更为重要。

Curry的工作对组合子逻辑有着非常大的影响，几乎成为了这个领域最闪耀的名字。Curry在数理逻辑方面的工作颇丰，奠定了后来函数式编程语言的基础。著名函数式编程语言Haskell以Haskell Curry的名字命名，就是为了纪念他的贡献。

SKI组合子演算

SKI组合子演算是一套符号系统，其中的所有函数可以只使用S、K、I的字母表和圆括号（分组符号）来表达，**仅仅需要这样几个简单的组合子相互应用，就可以达到图灵完备**。函数应用的概念详见后一节，这里可以简单地认为就是在形式把参数上代入函数的意思。我们约定组合子是左结合的，这样可以省略圆括号。

S是代换运算，它接受3个参数，分别将第一和第二参数应用于第三参数，然后将第一参数的应用结果应用到第二参数的应用结果上。如果将它写成常规的写法，就是 $S(x, y, z) = y(z)(x(z))$ ，就是将第三个参数代入第一个参数，然后结果代入第三个参数代入第二个参数生成的函数，和前面“应用”的意思是一致的。

$$S \ x \ y \ z \rightarrow x \ z \ (y \ z)$$

K是投影运算，它接受两个参数，返回第一个。

$$K \ x \ y \rightarrow x$$

I是恒等运算，它接受一个参数，返回这个参数。I也可以定义为 $S \ K \ K$ ，因为 $I = x \rightarrow ((S \ K \ K) \ x) = x \rightarrow (S \ K \ K \ x) = x \rightarrow (K \ x \ (K \ x)) = x \rightarrow x$ ，所以只用SK的组合子演算系统依然是完备的。

$$I \ x \rightarrow x$$

不妨和原始递归函数比较一下，看看有哪些相似和不同之处。

以布尔代数为例，我们来看一看SKI组合子的能力：我们定义 $T=K$ ， $F=KI$ ，实际上就是投影到第一参数和投影到第二参数的组合子。

```

T x y = K x y = x
F x y = K I x y = (K I x) y = I y = y

```

我们将这两个组合子的含义视为真 (True) 和假 (False)，然后我们就可以定义出布尔运算，其中OR是中缀运算符，NOT和AND是后缀运算符。

```

NOT = F T
OR  = T = K
AND = F = K I

```

可以验证，真值表是和我们通常的布尔运算一致的。

```

T NOT = T F T = F
F NOT = F F T = T

T OR T = T T T = T
T OR F = T T F = T
F OR T = F T T = T
F OR F = F T F = F

T T AND = T T F = T
T F AND = T F F = F
F T AND = F T F = F
F F AND = F F F = F

```

这里，对这方面的逻辑系统不熟悉的读者，应该会注意到一个概念，这里的**值**和**函数**事实上是等同的。的确，SKI组合子演算和 λ -演算都是侧重于**函数**而非**集合**的演算系统，它们所有的值都是函数。

1930s λ演算的诞生

λ-演算的早期历史

λ演算由Alonzo Church在1928年发明。1928年，Church试图建立一个形式系统，目的是，提供一个对于Russell的类型论和Zermelo的集合论更为自然的逻辑基础。而且，他希望这个形式系统**不包含自由变元**。

Church选择将这个系统的概念**建立在函数而非集合**之上，他引入了几个核心概念：

1. 函数抽象 $\lambda x[M]$ （我们现在记为 $\lambda x. M$ ）
2. 函数应用 $\{F\}(X)$ （我们现在记为 (FX) ）

加上“变量”的概念，这就是λ演算的基础元素。

为什么Church选择了“λ”作为符号？

1964年，他本人声称，这来源于Russell和Whitehead的类抽象符号 $\hat{x}$ 。他将 $\hat{x}$ 换成 $\wedge x$ ，以将函数抽象与之区分，然后为了印刷方便，把 $\wedge$ 换成了 λ 。不过，后来他又说这其实只是个偶然，他刚好需要一个符号并想到了这个而已。

Church不是第一个引入函数抽象记号的人，但是他是第一个阐明记号的转换规则，并深入分析的人。在1933年的论文中，他阐述了他设计的系统。论文末尾引入了我们现在称为丘奇数的概念，这可以在λ演算系统中定义自然数，由此可一瞥λ演算的能力：

$$1 \equiv \lambda xy. xy, \text{ Succ} \equiv \lambda xyz. y(xyz), n =_{\beta} \lambda xy. (x(\cdots)xy \cdots) (\text{. 后的 } x \text{ 出现 } n \text{ 次})$$

1931-1934年，Church与学生Stephen Kleene和Barkley Rosser一起发展了相关理论。他们失望地发现，1933年Church的论文中的逻辑系统是不完备的（哥德尔定理在同期发表）。但是，在其之下的纯λ演算系统，能力出人意料地强大。

“出人意料地” 源自 "Kleene was surprised" [Kleene, 1981, p.54].

题外话，正则表达式里面的 Kleene star $*$ 和对应的 Kleene 闭包运算就是以 Stephen Kleene 命名的。

λ 演算系统与组合子逻辑的关系也被阐明，它们是等价的。更进一步地， **λ -可定义函数类被证明和 Herbrand-Gödel 递归可计算函数类、Turing 可计算函数类等价。**

Alan Turing 在同期也独立证明了类似的结果。而且，Turing 给出了**不动点组合子**的概念。这在当时未引起注意，但是在后来被证明非常重要。Turing 给出的是 $(\lambda xy. y(xxy))(\lambda xy. y(xxy))$ ，后来常常被称为 Curry Y 组合子的是 $\lambda x. (\lambda y. x(yy))(\lambda y. x(yy))$

Y 不动点组合子有什么用？

在正统的 Lambda 演算里，函数全部是没有名字的，因此递归函数无法实现。而利用 Y 不动点组合子，我们可以实现递归。

这使得 Lambda 表达式真的不需要名字了。

参考：

[Y 不动点组合子用在哪里？](#)

λ -演算的基本概念

定义

λ -演算是由特定形式的语法所组成的一种语言，它侧重于将参数映射为结果的函数规则，而忽略函数的命名。 λ -记法就是一种表示匿名函数的符号。也因为这个原因，现代程序语言中的“lambda 表达式”一词往往和“匿名函数”是同义的。例如，对于函数

$$\forall x \in A, f(x) = x$$

用 λ -记法，我们记成

$$\lambda x. x$$

λ-演算系统从一个**标识符**的可数无穷集合开始，在λ-演算中只有三种合法表达式（**λ-表达式** 或 **λ-项**）存在：

1. 变量(Variable) / “标识符引用”：

诸如 x , y 这样的，在标识符集合里面的元素，我们称为变量。这就是一个名字，用于表示参数或者一个值。

注意，原始的λ演算是没有类型的，也没有变量定义的概念。这意味着，如果不赋予它特别的含义（如人为约定或者上下文约束），它就是一个符号而已，你用 x 命名和用 y 命名没有本质差别。这和前一节举的例子是一致的，我们**约定**了T和F两个组合子的**含义**，因为它的运算满足我们所需要的**性质**。

2. 抽象(Abstraction) / “函数定义”：

假定 x 是一个变量， M 是一个λ项，那么 $\lambda x.M$ 表示一个参数为 x ，返回值为 M 的函数的计算结果。换句话说，它就是代表输入参数 x ，输出 M 的函数。

对函数定义的表述到此为止，因为M是未定义的。此外，这时我们说：λ-表达式绑定了参数 x ，在M中所有的 x 的出现都被称为被表达式前面的那个λ所绑定，这个概念将在后续阐明。

3. 应用(Application):

假定 M , N 都是λ-项，将函数 M 应用于参数 N ，就是给函数 M 输入实参 N 。我们将它写成把函数值放到它的参数前面的形式，写作 $(M N)$ 。

如果严格地定义标识符集合，将上述三条规则形式化地表述并使用归纳定义，可以得到λ-表达式的定义。

$$\begin{aligned} M &= X \\ &\quad | (\lambda X. M) \\ &\quad | (M N) \\ X &= \text{a variable: } x, y, \dots \end{aligned}$$

λ-表达式的表达形式很容易产生大量括号，影响阅读。为此，我们约定：

1. 应用是左结合的： $f x y = (f x) y$ 。
2. 应用的优先级比抽象高： $\lambda x.MN$ 代表的是 $(\lambda x.(MN))$ 而非 $((\lambda x.M) N)$

例如，我们有变量 x ，对于 $f(x)+2$ ，可以表达为抽象 $\lambda x. x+2$ ，而对于 $f(3)+2$ ，则可表达为应用 $(\lambda x. x+2) 3$ 。

笑话一则：为什么要省略括号？

一个黑客说自己从NASA的主机盗取了一个用Lisp编写的核心程序，为了证明自己说的是真话，他打印出了程序的最后一页——全是右括号。

柯里化

在上面那个例子中，我们发现 $+$ 还没有定义，而加法是一个二元函数。而函数抽象的定义中，只允许一个参数。通过所谓的柯里化(Currying)，我们可以突破这个限制

对于 $x+y$ ，我们记成

```
 $\lambda x y . + x y$ 
```

这可以理解为

```
 $\lambda x . (\lambda y . + x y)$ 
```

因为 λ -演算中，函数就是值，我们可以写一个单参数函数，返回一个带一个参数的函数。通过柯里化，我们将多参数函数转换成了一系列的单参数函数。

```
currying:  $\lambda x y . xy = \lambda x . (\lambda y . xy)$ 
```

形式化地说，对于函数 $f: (X \times Y) \rightarrow N$ ，有 $\text{curry}(f): X \rightarrow (Y \rightarrow N)$

通过柯里化，我们在前面的约定中增加一条：

- 连续不断的 λ 表达式可以合并表示。例如 $\lambda xyz . M = (\lambda x . (\lambda y . (\lambda z . M)))$ ，相当于多参数函数。

自由变量和绑定变量

在表达式 $(\lambda x . (\lambda y . xy))y$ 中，很明显地，里面的 y 和外面的 y 是不一样的，这就是所谓的约束(bound)的概念。在一个 λ -项中，变量要么是自由出现的，要么是被一个 λ 符号绑定的，归纳地定义就是：

1. 在表达式 x 中, 如果 x 本身就是一个变量, 那么 x 就是一个单独的自由出现。
2. 在表达式 $\lambda x. E$ 中, 自由出现就是 E 中所有的除了 x 的自由出现。这种情况下, 在 E 中所有 x 的出现都称为被表达式中 x 前面的那个 λ 所绑定。
3. 在表达式 $(M N)$ 中, 变量的自由出现就是 M 和 N 中所有变量的自由出现。

自由出现的变量又称**自由变量**, 相应地被 λ 符号绑定的变量称为**绑定变量**, 也叫**约束变量**。直观地说, 只要 x 在任意一个 $(\lambda x. _)$ 表达式之外, 那么这个 x 就是自由变量。

我们用 FV (Free Variable) 来表示一个 lambda 表达式中所有自由变量的集合, 定义如下:

$$\begin{aligned} FV(X) &= \{X\} \\ FV((\lambda X. M)) &= FV(M) \setminus \{X\} \\ FV(M_1 M_2) &= FV(M_1) \cup FV(M_2) \end{aligned}$$

下面是一些例子

$$\begin{aligned} FV(x) &= \{x\} \\ FV((x(yx))) &= \{x, y\} \\ FV((\lambda x. (xy))) &= \{y\} \\ FV((z(\lambda z. z))) &= \{z\} \\ FV((\lambda y. y)(\lambda x. xy)) &= FV(\lambda y. y) \cup FV(\lambda x. xy) = \emptyset \cup \{y\} = y \\ FV(\lambda x. (\lambda y. xyz)) &= FV(\lambda y. xyz) \setminus x = \{x, z\} \setminus x = \{z\} \end{aligned}$$

由于每个 lambda 函数都只有一个参数, 因此也只有一个绑定变量, 这个绑定变量随着形参的变化而变化。例如, 对于 $\lambda f. \lambda x. fx$, 我们逐一拆开函数体: $\lambda f. \lambda x. fx = \lambda f. E, E = \lambda x. fx = \lambda x. A, A = fx = A_1 A_2, A_1 = f, A_2 = x$, 所以, 在 A 中, f 和 x 都是自由出现的; 在 E 中, x 是绑定 λx 的 λ 的; 整个公式中, f 是绑定第一个 λf 的 λ 的。

1940s-1950s的发展

30年代后的一些历史

在30年代之后的许多年， λ 演算和组合子逻辑都对多数逻辑学家没有吸引力，使得它们的理论进展缓慢。其中的一个原因是，Turing的机器模型更显然、更易于理解，甚至Kleene在他的后续计算理论的研究中也更偏好其它的形式系统。但是二者都在逐步完善，包括了 λ 演算的类型系统、二者的规约理论等。

λ 演算的 β -规约的相关理论就是在这个时期逐渐完善的。Curry等人也将组合子逻辑的weak reduction发展起来。不过，人们发现，即使有规约规则存在，用 λ 演算似乎总是比组合子逻辑容易，尽管后者的形式更简单。

逻辑学给我们的启示

形式系统越简单，推导起来越复杂。简单的系统只是理论上好用而已。

比如命题演算的自然推理形式系统N，与形式系统P：前者符合人的推理直觉，好写好理解，但是规则稍复杂；后者规则简单，但那种消解证明法几乎完全与人的逻辑思考方式相悖，让人很难写。

参考：[如何理解面向组合子编程？- 何幻的回答](#)

不过，在另一方面，Church写了一本可读性很好的 λ 演算的入门教科书。Paul Rosenbloom在他的逻辑教科书中专门划出了一个章节，介绍组合子逻辑。两个学科在技术方面也都取得了进展，这也为后来的发展奠定了基础。

由此可见，写一本好的教科书是有多重要啊！

λ 演算的变换规则

λ -演算有 α , β , η 三条变换规则，其中 η 是可选的。

形式化地，有如下表述。

$$\begin{aligned}
(\lambda X_1. M) &\rightarrow_{\alpha} (\lambda X_2. M[X_1 \leftarrow X_2]) \quad \text{where } X_2 \notin \mathcal{FV}(M) \\
((\lambda X. M_1) M_2) &\rightarrow_{\beta} M_1[X \leftarrow M_2] \\
(\lambda X. (MX)) &\rightarrow_{\eta} M \quad \text{where } X \notin \mathcal{FV}(M)
\end{aligned}$$

下面一一阐述。

1. α -转换：对绑定变量重命名，这条规则用来避免名称冲突。规则如下：

$$(\lambda x. M[x]) \rightarrow (\lambda y. M[y])$$

约束：y不是M的自由变量。也就是说，y没有在M中自由出现，并且只要y替换M中的x，都不会被M中的一个 λ 绑定。

这条规则是说， λ 演算中约束变量的名称是不重要的，例如 $\lambda x.x$ 和 $\lambda y.y$ 是相同的函数。因此，将某个函数中的所有约束变量全部换名是可以的，只要不改变表达式含义。例如， $\lambda x. \lambda x. x \rightarrow \lambda y. \lambda x. x$ 是合法的 α 转换，而 $\lambda x. \lambda x. x \rightarrow \lambda y. \lambda x. y$ 不是，因为转换后的y被内层的 λ 绑定。 $\lambda x. \lambda y. x \rightarrow \lambda y. \lambda y. y$ 同样是不合法的，因为在x在函数 $\lambda y.x$ 中有自由出现。

2. β -规约：在抽象化的函数定义体中，以参数表达式代替绑定变量。换句话说，它是函数代入这个概念的形式化表示，正是因为我们能进行函数代入，我们才能将函数应用规约成值。规则如下：

$$((\lambda x. M) E) \rightarrow (M[x := E])$$

约束： E 中的所有变量的自由出现都在 $M[x := E]$ 中保持自由出现。

这个规则的约束有点复杂，参考[某些资料](#)的做法，我们也可以将它的定义拆开来分别定义两种过程：

对于一个函数应用 $(\lambda x. x) y$ ，它意为将函数 $\lambda x. x$ 应用于 y 。 $(\lambda x. x) y \rightarrow x[x := y]$ 一步叫做 β -归约，**这一步可以认为没有限制**，只是形式上的写法变换， $x[x := y] = y$ 一步称作**替换** $[- \leftarrow -]$ 。注意，替换操作 $:=$ 的记法有很多，常见的有左箭头 $\leftarrow$ ，左斜杠/等。替换是一种**等值变换**，只能将**自由变量**的实例替换成参数表达式：

$$\begin{aligned}
X_1 [X_1 \leftarrow M] &= M \\
X_2 [X_1 \leftarrow M] &= X_2 \\
&\text{where } X_1 \neq X_2 \\
(\lambda X_1. M_1) [X_1 \leftarrow M_2] &= (\lambda X_1. M_1) \\
(\lambda X_1. M_1) [X_2 \leftarrow M_2] &= (\lambda X_3. M_1 [X_1 \leftarrow X_3] [X_2 \leftarrow M_2]) \\
&\text{where } X_1 \neq X_2, X_3 \notin \mathcal{FV}(M_2) \\
&\text{and } X_3 \notin \mathcal{FV}(M_1) \setminus \{X_1\} \\
(M_1 M_2) [X \leftarrow M_3] &= (M_1 [X \leftarrow M_3] M_2 [X \leftarrow M_3])
\end{aligned}$$

注意，倒数第二条规则并没有限制 $X_3 \neq X_1$ ，如果 $X_1 \notin \mathcal{FV}(M_2)$ ，二者可以相同。

举例：

```

(λx. λy. y x) (λz. u) → λy. y (λz. u) 将x代入了λy. y

(λx. x x) (λz. u) → (λz. u) (λz. u) 将λz. u代入了x x

(λy. y a) ((λx. x) (λz. (λu. u) z)) → (λy. y a) (λz. (λu. u) z) 将(λz. (λu. u) z)代入x

(λy. y a) ((λx. x) (λz. (λu. u) z)) → (λy. y a) ((λx. x) (λz. z)) 将z代入(λu. u)

(λy. y a) ((λx. x) (λz. (λu. u) z)) → ((λx. x) (λz. (λu. u) z)) a 将(λx. x) (λz. (λu. u) z)代入y

```

一个运用了两种规则的简单例子，其中的 α 转换是因为绑定变量 y 将与自由变量 x （将被替换为 y ）冲突，所以更名为 z 。

$$\begin{aligned}
&(\lambda x. (\lambda y. xy))y \\
&\rightarrow_{\beta} (\lambda y. xy)[x \leftarrow y] \\
&\rightarrow_{\alpha} (\lambda z. xz)[x \leftarrow y] \\
&= \lambda z. yz
\end{aligned}$$

3. η -规约（可选）：两个函数若对于相同的输入产生相同的输出，则认为它们可以互相规约。这条规则表达的是外延性的概念。

$$\lambda x. Mx = M$$

约束： x 不是 M 的自由变量。

这条规则可用前两条规则导出，考虑 $\lambda x.M\ x$ ，将它应用于任意一个参数上，比如 $(\lambda x.M\ x)\ N$ ，经过 β -规约，发现它等价于 $M\ N$ ，那么，这意味着 $\lambda x.M\ x = M$ 。也就是说， $\lambda x.M\ x$ 的表达式 M 不包含绑定变量 x 的函数抽象，它是冗余的，等价于 M 。

λ -演算在这些规则之下构成了一个图灵完备的计算系统。例如，我们可以用丘奇数定义自然数：

```
0 =  $\lambda f.\lambda x.x$ 
1 =  $\lambda f.\lambda x.f\ x$ 
2 =  $\lambda f.\lambda x.f\ (f\ x)$ 
3 =  $\lambda f.\lambda x.f\ (f\ (f\ x))$ 
...
```

定义后继函数，返回 $n+1$ ：

```
SUCC =  $\lambda n.\lambda f.\lambda x.f\ (n\ f\ x)$ 
```

例如

```
SUCC 0
= ( $\lambda n.\lambda f.\lambda x.f\ (n\ f\ x)$ )  $\lambda f.\lambda x.x$ 
→ ( $\lambda f.\lambda x.f\ (n\ f\ x)$ ) [ $n := \lambda f.\lambda x.x$ ] // beta
→ ( $\lambda g.\lambda y.g\ (n\ g\ y)$ ) [ $n := \lambda f.\lambda x.x$ ] // alpha
→  $\lambda g.\lambda y.g\ ((\lambda f.\lambda x.x)\ g\ y)$  // substitute
→  $\lambda g.\lambda y.g\ (x[f := g, x := y])$  // beta
→  $\lambda g.\lambda y.g\ y$  // substitute
= 1
```

在此基础上，我们可以定义出加法，进而定义乘法，以及逻辑谓词等等。加上用前面提到过的Y不动点组合子实现递归，这几乎可以说明它是图灵完备的了。

```
PLUS =  $\lambda m.\lambda n.\lambda f.\lambda x.m\ f\ (n\ f\ x)$ 
MULT =  $\lambda m.\lambda n.m\ (PLUS\ n)\ 0$ 
PRED =  $\lambda n.\lambda f.\lambda x.n\ (\lambda g.\lambda h.h\ (g\ f))\ (\lambda u.x)\ (\lambda u.u)$ 
TRUE =  $\lambda x.\lambda y.x$ 
FALSE =  $\lambda x.\lambda y.y$ 
AND =  $\lambda p\ q.p\ q\ FALSE$ 
OR =  $\lambda p\ q.p\ TRUE\ q$ 
NOT =  $\lambda p.p\ FALSE\ TRUE$ 
IFTHENELSE =  $\lambda p\ x\ y.p\ x\ y$ 
Y =  $\lambda f.(\lambda x.f\ (x\ x))\ (\lambda x.f\ (x\ x))$ 
```

1960s 编程语言

进入60年代， λ 演算和组合子逻辑开始吸引逻辑学家社群之外的人们——主要是后来被称为「计算机科学家」的群体。

John McCarthy与Lisp

John McCarthy从1956至1960开发了**LISP编程语言**。LISP是被作为一个带函数抽象功能的表处理语言来开发的。McCarthy的目标是将LISP应用于非数值计算的领域，一门新生的学科——人工智能。尽管LISP不直接基于 λ 演算，但是它的抽象记号直接用lambda来命名。McCarthy对编程语言的理论有很大影响，比较著名的是**条件表达式(if-then-else)** 及其递归应用，所谓**McCarthy Formalism**。

LISP与AI

翻开任何一本介绍各种编程语言的书，每当说到Lisp，通常的话就是“LISP是适合人工智能（AI）的语言”。但是，为什么我从来都不觉得LISP“适合”AI？后来查了一下资料果然也有人这么认为。

Lisp在上世纪经常被用于AI领域，其实就是因为McCarthy恰好是AI之父，然后他和他的学生、同事都用Lisp写程序而已。Lisp是通用语言，当然不止可以用于AI，当年甚至有人做过LISP Machine. 真要说Lisp与AI的关系的话，可能是因为链表比数组更适合当时符号主义AI的表达需求，所以McCarthy选择了链表作为Lisp的基础数据结构。需求驱动发明，毕竟，FORTRAN也是被数值计算的需求所推动的嘛。

可以看看**ANSI Common Lisp**作者Paul Graham是怎么说的：

Lisp was invented by John McCarthy, who also invented the term "artificial intelligence". His students and colleagues wrote their programs in Lisp, and so it began to be spoken of as an AI language. This line was taken up and repeated so

often during the brief AI boom in the 1980s that it became almost an institution.

-- Paul Graham *On Lisp*

参考：

[LISP 语言是怎么来的—LISP 和 AI 的青梅竹马 A](#)

[LISP 语言是怎么来的—LISP 和 AI 的青梅竹马 B](#)

[为什么 Lisp 大多用于人工智能领域？](#)

Peter Landin与Algol 60

在60年代的早期，Peter Landin提议用 λ -项来构建Algol 60. 相比Lisp为了解释器实现方便而使用动态绑定的做法，Algol的块结构和 λ 演算的命名处理方式更加契合。

事实上，Landin的工作使得 λ 演算本身成为编程语言成为可能。1963年，他描述了一种规约 λ 项的抽象机SECD-machine。这个机器的运算顺序的转换规则就是现代编程语言中的 **传值调用(call by value)**。传值调用是非常重要的参数调用概念。

他还在SECD-machine的文章中定义了 **闭包(closure)** 的概念，他用此指代自由变量由其文法环境所约束的 λ 表达式。我们现在通常认为，**引用了自由变量的函数，或者说能够捕获其环境中的变量的函数，称为闭包**。闭包和匿名函数的概念对编程语言影响甚广。

ALGOL的历史影响

ALGOL系列编程语言是早期编程语言的代表作之一，发展于1950年代中期，对许多其它编程语言产生了重大影响。在ALGOL 60报告中引入的巴克斯范式BNF，几乎成为后世所有语言设计文法的描述规则。

大多数近代编程语言皆使用类似ALGOL的语法。它影响了PL/I、Simula（和Smalltalk一同被认为面向对象的起源，深深影响了

C++)、BCPL (影响了B)、B (C语言的前身)、Pascal (作者Wirth原本是想设计“Algol W”)、C (影响至今) 等一大批语言。函数式语言Scheme也受到ALGOL影响。

ALGOL本身并没有像同期的FORTRAN等语言沿用至今 (我觉得, 不是因为数值计算库高度依赖FORTRAN写的祖传代码, FORTRAN这个古董早也应该下葬了)。不过, 即使在2020年的bash(Bourne Again SHell, 源自1979年的Bourne Shell), 你仍然可以看到Algol的关键字遗迹——像是if - fi, case - esac这种奇特的、用反写关键字表示语句块结束的写法, 就是来源于ALGOL 68.

参考:

[ALGOL - Wikipedia](#)

[What does “esac” mean at the end of a bash case statement? Is it required?](#)

Corrado Böhm的贡献

λ演算和组合子逻辑开始向编程语言贡献力量。但是, 这时候它们的影响力其实还很小, 直到Corrado Böhm等人的工作改变了这一切。

1960-61年, Böhm和Gross一起学习组合子、λ演算和递归函数理论。到1964年, 他们意识到, λ演算和组合子可以直接作为“一类非常灵活的机器、程序和算法的描述语言”。他们设计了CUCH语言, 相关研究涵盖了λ演算和组合子逻辑的编程语言语法理论。这也引发了其他研究者的兴趣, 并在60年代后取得了很多进展, 尤其是编程语言的语法理论上。

Böhm最著名的成果是[结构化程序定理](#), 引出了我们现在所说的“[结构化编程](#)”。简要地说, 对过程化编程有着重大影响的“结构化编程”概念是在60年代末70年代初发展起来的。两个关键的时间点是Böhm和Jacopini的1966年论文, 和1968年Edsger W. Dijkstra著名的“GOTO有害论”。

早期的函数式编程语言

McCarthy在50年代末设计了Lisp的早期版本，使用了Church的 λ 符号标识函数，Peter Landin的Iswin语言和Iverson的APL语言都对函数式编程范式都起到了推动作用。但是，这时候还没有函数式编程的概念。

LISP有很多优异的特性：

1. 垃圾回收。
2. 用闭包实现静态作用域。例如，`let x = 3 in f =  $\lambda$  y.x + y` 函数 `f` 不是闭合的，因为它有自由变量 `x`。为了使用 `f`，它必须被一个环境所约束，并得到 `x` 的值。
3. 条件表达式和用条件表达式实现递归函数。
4. 对列表的高阶操作，例如 `mapcar` 函数。

现在，几乎所有的函数式编程语言使用垃圾回收来进行内存管理，而且使用闭包实现静态作用域。条件语句几乎出现在了每一个现代高级语言，而高阶函数被视为函数式语言的核心特性之一。

事实上，在90年代后，垃圾回收的应用变得尤为广泛。因为，硬件的性能已经到了一个临界点，设计者可以通过牺牲性能、简化语言来提高程序员的生产力。著名的例子包括Java, Python, Ruby等等。

之后的历史

在此之后，对 λ 演算和编程语言的大量研究开始考虑类型系统的问题，导致了类型理论的各类定义研究，以至于高阶类型理论的研究。诸如和类型（Sum Type，常称为Tagged Union，常常以variant的名称出现）、积类型（Product Type，几乎所有支持自定义类型的现代语言都有，例如C语言的struct）、交集类型、递归类型等更是深深地影响了现代的编程语言。学者们还着手与将 λ 演算严格化，建立了良好定义的数学模型。这方面的理论非常艰深，一个快速的科普可参考[Belleve的知乎答案](#)。

1970s & 2010+ 函数式编程语言

函数式编程的概念

Backus在他的1978年图灵奖演讲中定义了函数式编程语言（Functional Programming, FP）。Backus还论述了为什么命令式编程“不好的”，为什么函数式编程是“好”的。

FP的核心概念是**高阶函数**(Higher order function)，就是将函数视为一阶公民，函数将函数视为参数，并返回函数。这对表达函数的组合有巨大帮助。和 λ 演算的定义比较一下——这不正是 λ 演算在做的事情嘛！函数式编程的基础正是 λ 演算，所有的函数式编程语言都具有高阶函数。

除此之外，函数式编程的两大特色是纯函数(Pure function)和惰性计算(Lazy evaluation, 又称延迟计算)。不过，不同的FP语言对此的态度有所不同。

纯函数是无副作用的函数，对相同的参数的返回值永远一致。“副作用”指的是操作符、函数或表达式对局部环境之外的状态变量的修改，例如修改全局变量，修改局部静态变量，修改传引用的参数，进行I/O操作等等。纯函数更接近“数学意义上的函数”。

惰性计算最早是Christopher Wadsworth在开发一个 λ 演算计算系统时发明，这是一种将表达式的计算延迟到需要表达式的值的时候的计算策略，又称**按需调用**(call-by-need)。

而函数式语言的**类型系统**往往也很有特色，其发展出的一整套方案也深深影响了命令式语言的发展。

Lisp的独特魅力

方言

Lisp不仅是一门函数式编程语言，还是现今第二悠久而仍然广泛使用的高级语言（只有FORTRAN比它早一年）。在此过程中，Lisp演化出了数不胜数的方言。

它们都以S-表达式为核心，具体实现上大相径庭，可能连关键字都不一样（定义函数的关键字，Common Lisp用 `defun`，而Scheme用 `define`）。现在仍然活跃的包括

1. Common Lisp
2. Scheme
3. Emacs Lisp
4. Clojure
5. Racket

其中最著名的两个方言是 `Common Lisp` 和 `Scheme`。

Common Lisp的教程可阅读[ANSI Common Lisp](#)。Scheme入门可参考[Teach Yourself Scheme in Fixnum Days](#)。此外，还可以阅读大名鼎鼎的[Structure and Interpretation of Computer Programs](#)来学习。

Lisp的方言为什么这么多？

Lisp方言众多，其中一个原因是它很灵活（Lisp的宏可以操纵语法树，制定自己的语法），还有一个重要原因是Lisp的标准化太迟了，Lisp在1958年发明，而ANSI Common Lisp标准直到1994年才出版，相隔了36年（相比之下，K&R C和ANSI C只相差11年）。就连最著名的Common Lisp和Scheme，其本身的实现还各有一批方言..... 由于大量 Lisp 方言的存在，导致如今的社区分裂得很厉害。

语言**标准**的重要性不言而喻。

S-表达式

Lisp是一种基于表达式的语言，其“语句”和“表达式”并无不同，所有的代码和数据都是表达式，当表达式被计算的时候，它产生值。

Lisp的核心概念是**S-表达式**（Symboic expression, S-Expression，符号表达式），这是一种用来表达嵌套**列表**的记法。所有Lisp语句都是S-表达式。传统上，S-表达式递归地定义为

1. 一个原子(atom)
2. 一个 `(x . y)` 形式的表达式，其中x和y都是S-表达式

很明显，这意味着S-表达式都是二叉树。而“原子”可以是数值、字符串、变量标识符等等。

现代的S-表达式往往还加入了 `(x y z)` 的记法，代表 `(x . (y . (z . NIL)))`，其中 `NIL` 是特殊的列表终止对象。

在基于S-表达式的同时，Lisp还使用前缀表示法。这样的语法和树形结构使得Lisp的解析器非常好写。

举一个Common Lisp的例子：

```
(defun factorial (x)
  (if (zerop x) 1
      (* x (factorial (- x 1)))))
```

举一个Scheme的例子：

```
(define (factorial x)
  (if (zero? x) 1
      (* x (factorial (- x 1)))))
```

从上面的例子可以发现：

1. Lisp的函数也是用S-表达式表示成列表的。函数在Lisp中是一等公民，可以被当成值来传递。
2. 和Algol系语言不同，Lisp没有操作符的概念，操作符也是函数。算术运算符在Lisp中是一种变参函数。

注意，Lisp的 `and` / `or` 这样的逻辑运算符仍然具有短路能力。一般的函数是做不到短路能力的，Lisp之所以能做到，是因为Lisp具有强大的宏。

Lisp与Web

以当今Web开发的流行程度来看，Javascript可以算是「人生赢家」了。不过，正所谓「Javascript和Java的关系，就像雷锋塔和雷锋的关系一样」，JS原作者Brendan Eich最初加入网景的原因，其实是网景给他画的大饼「在浏览器里写 Scheme」。后来，因为时间太紧（仅仅10天），加上网景与Sun的合作，才导致了JS设计成后来的样子。

尽管 Scheme 的诱惑已经不再，Brendan Eich 仍然发现 Lisp 式的函数一等公民概念很有吸引力。Javascript函数一等公民对应的这套工具深受 Scheme 习惯用法影响。

现在，S-表达式不止在Lisp中使用，当今Web的未来之星 WebAssembly，其字节码的文本表示就是S-表达式。

参考

Javascript : the first 20 years

<https://zenodo.org/record/3707008>

(译文 <https://zhuanlan.zhihu.com/p/122334333>)

WebAssembly - Wikipedia <https://en.wikipedia.org/wiki/WebAssembly>

高阶函数

S-表达式的处理对象是**列表**（别忘了，LISP原意是**LISt Processor**）。因此，Lisp广泛使用与列表迭代相关的**高阶函数**。在命令式的编程语言往往需要显式循环的地方，Lisp使用高阶函数来完成。

最有名的高阶函数可能是 `map` 和 `reduce` 了，这两个高阶函数启发了Google的研究人员设计出了MapReduce数据处理模型，催生了Apache Hadoop.

Scheme中有一个叫做 `map` 的函数，在Common Lisp中叫 `mapcar`。给定一个函数和若干列表，`map` 将函数顺序应用到列表的元素上，并将结果收集成一个新的列表：

```
(map + '(1 2 3 4 5) '(10 20 30 40 50))
```

结果是 `(11 22 33 44 55)`

`reduce` 将二元函数相继应用于列表的一对对元素，收集得到结果。

```
(reduce-left + 0 '(1 4 9 16))
```

结果是 `30` .

高阶函数的一大优点是可组合性。在数据处理方面，这有着天然的优势。

举个例子：C#的LINQ被认为属于函数式编程的风格。下面是一段使用了LINQ的C#代码，其功能是读取文件中的每一行并做归类统计。

```
File.ReadAllLines(filepath)
    .SelectMany(line => line.Split(' '))
    .GroupBy(word => word)
    .Select(g => new { Word = g.Key, Count = g.Count() })
    .Where(t => t.Count > 100)
    .OrderByDescending(t => t.Count)
    .ForEach(t => Console.WriteLine("{0}: {1}", t.Word, t.Count));
```

这里的Linq扩展方法 `SelectMany / GroupBy / Select / Where / OrderByDescending / ForEach` 都是高阶函数，它接受一个函数，并将它们应用到这个序列的每一个元素上。

设想一下，如果用命令式的写法，是否还能像这样的扁平、逻辑清晰？要写多少层的循环、条件语句，要设计多少个中间变量？如果我突然想增加、修改条件，又要移动多少的代码？这些问题的核心，就是所谓的“可组合性”。

宏

Scheme作者对于设计程序语言的态度

“Programming languages should be designed not by piling feature on top of feature, but by removing the weaknesses and restrictions that make additional features appear necessary.”

— *Revised⁶ Report on the Algorithmic Language Scheme*

Lisp的另一大特色是宏。因为Lisp的语法结构中，有 `quote` 就是数据，没 `quote` 就是程序。这种“数据即程序，程序即数据”的概念，使得它的宏非常灵活。

宏是也是一种代码，是操纵代码变换的代码，即所谓的“元编程”：对语法树进行变换，扩展编译器的功能。宏的最大用途是自定义语法扩展或领域特定语言(Domain Specific Language, DSL)，这给了我们自定义语言特性的能力。

下面的例子来源于Stack Overflow的一个问答：在Python中，我们有列表生成式(List comprehension expressions)。在Python 1.5的时候，Python还没有

这个功能。但是对于Lisp来说，我们想要一个语法就是一个宏的事：

```
(defun range-helper (x)
  (if (= x 0)
      (list x)
      (cons x (range-helper (- x 1)))))

(defun range (x)
  (reverse (range-helper (- x 1))))

(defmacro lcomp (expression for var in list conditional conditional-
test)
  ;; create a unique variable name for the result
  (let ((result (gensym)))
    ;; the arguments are really code so we can substitute them
    ;; store nil in the unique variable name generated above
    `(let ((,result nil))
      ;; var is a variable name
      ;; list is the list literal we are suppose to iterate over
      (loop for ,var in ,list
            ;; conditional is if or unless
            ;; conditional-test is (= (mod x 2) 0) in our examples
            ,conditional ,conditional-test
            ;; and this is the action from the earlier lisp example
            ;; result = result + [x] in python
            do (setq ,result (append ,result (list ,expression))))
      ;; return the result
      ,result)))
```

然后我们就可以这么写

```
CL-USER> (lcomp x for x in (range 10) if (= (mod x 2) 0))
(0 2 4 6 8)
```

在上面的代码中，我们可以发现Common Lisp已经预置了一个 `loop` 循环。实际上，`loop` 也是一个宏，而不是保留字。

C语言的宏

C语言的宏和Lisp的宏有很大不同。C语言的宏只是简单的文本替换，而不受词法作用域约束，很容易造成名称泄露。

思考一下：如果在C语言中 `for` 和 `while` 不是保留字，那么怎么写循环？

ML的类型系统

ML开发于1970年中期，其特色是静态的类型系统（丘奇原始的 λ 演算是无类型的），代数数据类型和模式匹配。现代的ML系语言有OCaml和F#等。

静态类型系统为每个表达式赋予了**类型**，例如，表达式3是一个整数int类型，函数 `f =  $\lambda$  x. x + 1` 是一个从整数到整数的函数类型 `int->int`。通过类型系统，编译器能够在编译时检查类型是否匹配，以排查错误。

代数数据类型

代数数据类型(Algebraic Data Type) 是ML最受欢迎的特性之一，后来被引入大量程序语言。代数数据类型的“代数”指的是类型通过“代数”运算得到。ADT最基础的代数运算有两种：

1. 和 $A \mid B$ ：这个类型是A类型或者B类型，但是不能同时是A和B类型。
2. 积 $A * B$ ：这个类型是A类型和B类型的组合。

例如，在Standard ML中，如果我们想定义一个整数的树类型，我们可以这么定义包含整数的树：

```
datatype inttree = Empty | Node of int * int inttree list
```

它的含义是，`inttree` 要么是一个空节点，要么是一个包含了一个整数和一个 `inttree` 列表的节点。可以注意到的一点是，这样表达递归结构非常方便。

我们来看一个OCaml官方教程中的例子：

```
# type foo =  
  | Nothing  
  | Int of int  
  | Pair of int * int  
  | String of string;;  
type foo = Nothing | Int of int | Pair of int * int | String of string
```

值得注意的是，这样的表达非常接近**数学**而不是**机器**。直观上来说，这也是函数式编程和命令式编程的差别之一。

与上面等价的C语言代码大概要写成这个样子，一般称为Tagged Union. 不同的是，C的类型系统对于这个foo没有任何保证（说明：这里的type写成enum类型可能会好一点，但是C仍然不能约束强制类型转换。而且，当type == TYPE_INT的时候，写入foo.str并不能被编译器检测出来），而OCaml的类型系统可以检查出所有类型分支，并给出判断。这能够在编译期实现约束，消除错误。

```
struct foo {
    int type;
#define TYPE_INT 1
#define TYPE_PAIR_OF_INTS 2
#define TYPE_STRING 3
    union {
        int i;          // If type == TYPE_INT.
        int pair[2];    // If type == TYPE_PAIR_OF_INTS.
        char *str;      // If type == TYPE_STRING.
    } u;
};
```

和类型和积类型

通过积运算得到的类型称为**积类型**。积类型往往以**记录(record)**、**结构(struct)**等名称出现，出现在每一个支持用户自定义数据结构的现代语言中。

通过和运算得到的类型称为**和类型**。和类型则往往以**变体(variant)**、**带标签的联合体(tagged union)**、**可辨识的联合体(discriminated union)**等名称出现。这在一些语言中并不原生支持。和类型的一个实际应用就是实现类型范围在编译期确定的运行时多态，这样的开销往往比虚函数等动态分派手段要小。

参数多态和类型推导

ML允许**参数多态**，常常被现代语言称为**泛型**，这允许我们写出通用的函数和数据结构，举一个OCaml的二叉树例子：

```
# type 'a binary_tree =
    | Leaf of 'a
    | Tree of 'a binary_tree * 'a binary_tree;;
type 'a binary_tree = Leaf of 'a | Tree of 'a binary_tree * 'a binary_tree
```

这是一个通用的形式。用来存储整型的树是 `int binary_tree` 。同样我们可以定义存储字符串的数 `string binary_tree` 。

进一步地，ML实现了**类型推导**，这使得绝大部分类型能够从它们之间的关系自动地推导出来，大大提升了函数式编程语言的实用性。下面一个例子中，类型推导会自动为我们推导出叶节点的类型：

```
# Leaf "hello";  
- : string binary_tree = Leaf "hello"  
# Leaf 3.0;;  
- : float binary_tree = Leaf 3.
```

类型推导有什么用？

静态类型语言没有类型推导的话，将大大降低程序员的书写体验。

一些复杂的类型，虽然它的确有个类型，我们其实不关心它的类型细节，而更关心它代表的（语义上的）含义（动态类型为什么吸引人？）。在C++这种模板套着模板的情形下，这种问题更为严重。C++ 98没有类型推导，ISO C++标准委员会主席Herb Sutter在介绍C++ 11 auto时讲过一个笑话，“（演示了一个写错迭代器类型的例子）这个时候编译器就会报错，跟你说：「我知道你这边的类型写错了，而且我知道你应该写成什么，但我就是不告诉你，你赶紧给我改正。」”。我举一个例子：

```
std::vector<std::map<std::string, std::vector<int>>>> v;  
for(std::vector<std::map<std::string, std::vector<int>>>::  
iterator it = v.begin(); it != v.end(); it++)  
{  
    /* detail omitted */  
}
```

这里的迭代器类型冗长，但是我们其实根本不关心迭代器是什么具体类型，只关心它是个迭代器而已。而且编译器其实知道，它的类型应该是v.begin()的类型。所以，在C++ 11中，引入了自动类型推导 auto

```
std::vector<std::map<std::string, std::vector<int>>>> v;  
for(auto it = v.begin(); it != v.end(); it++)  
{
```

```
/* detail omitted */  
}
```

更重要的是，对于匿名函数(lambda)来说，我们实际上写不出它的类型，只能靠类型推导。在C++14中，我们甚至有了泛型lambda：

```
auto closure = [] (auto x, auto y) { return x * y; }
```

这里的closure的底层类型名称只有编译器才知道，所以closure的声明必须使用auto。

参数多态和类型推导是非常有用的特性，几乎是现代静态类型语言的标配功能了。二者搭配起来，能够使得静态类型的程序语言的体验趋近于动态类型的程序语言，而又不失类型系统的约束。

为什么我们需要静态类型系统？

自由地写动态类型语言不好吗，还完全不需要标注类型呢？

以下叙述来自 *Real World Haskell*：

“要理解静态类型的好处，可以用玩拼图的例子来打比方：在Haskell里，如果一块拼图的形状不正确，那么它就不能被使用。另一方面，动态类型的拼图全部都是1 x 1大小的正方形，这些拼图无论放在那里都可以匹配，为了验证这些拼图被放到了正确的地方，必须使用测试来进行检查。”

就本人写Rust的经历来说，我非常同意这个观点。静态类型系统在类型推导等一系列功能加持下，动态类型的主要优点几乎都能实现：

1. 动态类型不需要标注类型：在类型推导足够强大的情况下，静态类型系统根本不需要标注多少类型
2. 动态类型的多态性：在参数多态的条件下，静态类型系统写出通用的函数不是问题。而且，还可以通过具现化泛型(Reified Generics)的实现保留类型信息，使得编译器进一步优化（例如C++的模板，C#的泛型）。

而且不会出现动态类型的缺陷：

1. 动态类型的错误一般是运行时错误：在静态类型系统强约束的条件下，运行时错误几乎不会出现（Real World Haskell更是声称

Haskell “没有运行时错误”）。程序逻辑很可能一编译通过就是对的，大大节省编写和调试的时间。Python虽然写起来舒服，但是 `TypeError` 几乎是运行时错误最常见的一类了。

2. 动态类型语言类型推导的困难：静态类型系统的大多数推导规则都是编译期可以确定下来的。

这里的2可以进一步说明：

现代的IDE的代码补全和代码分析功能依赖于类型推导（IDE需要知道你在写的标识符是什么类型，才能推断出这个类型的方法、运算符、可以出现的上下文）。这在动态语言中难以进行，导致补全功能几乎失效。这使得现在的动态语言纷纷发展静态类型标注的方案，例如Python Type Hints和TypeScript。

更重要的是，动态类型的性能优化高度依赖于JIT，其中类型推导是重要一环。很多时候，JIT编译器需要猜测变量的类型，如果类型系统过于动态导致很难猜对，就麻烦了。在市场强烈需求的条件下，现在JavaScript的JIT的性能已经大幅提升，而Python这种过于动态的语言仍然存在很多障碍（我们现在有了PyPy，但是和CPython Runtime不兼容，使得大量Python库用不了，而**迄今为止所有试图兼容CPython的JIT实现全部都失败了**。当然，这其中很大程度上是CPython Runtime API设计糟糕，以及核心开发者不接受的因素）

2010年以后的新语言纷纷回归静态类型系统(Rust/Kotlin/Swift)，或者是没那么动态的动态类型系统(Julia)，或者给动态类型语言加可选的静态类型系统(TypeScript/Flow)，不是没有原因的。

模式匹配

给定一个类型的值，我们可以用**模式匹配**来判断它是ADT的具体哪一种。简单地说，我们可以将模式匹配 `match` 看成更强大的 `switch case`，所不同的是，`match` 允许匹配**类型**，这比一般类C语言中 `switch` 只能匹配值（甚至只允许整数常量）要强大太多。

在OCaml中定义一个表达式类型：

```
# type expr =  
  | Plus of expr * expr      (* means a + b *)
```

```

| Minus of expr * expr      (* means a - b *)
| Times of expr * expr      (* means a * b *)
| Divide of expr * expr     (* means a / b *)
| Value of string           (* "x", "y", "n", etc. *);;

type expr =
  Plus of expr * expr
| Minus of expr * expr
| Times of expr * expr
| Divide of expr * expr
| Value of string

```

让我们根据不同的类型实现不同的字符串转换：

```

# let rec to_string e =
  match e with
  | Plus (left, right) ->
    "(" ^ to_string left ^ " + " ^ to_string right ^ ")"
  | Minus (left, right) ->
    "(" ^ to_string left ^ " - " ^ to_string right ^ ")"
  | Times (left, right) ->
    "(" ^ to_string left ^ " * " ^ to_string right ^ ")"
  | Divide (left, right) ->
    "(" ^ to_string left ^ " / " ^ to_string right ^ ")"
  | Value v -> v;;
val to_string : expr -> string = <fun>

# let print_expr e =
  print_endline (to_string e);;
val print_expr : expr -> unit = <fun>

# print_expr (Times (Value "n", Plus (Value "x", Value "y")));;
(n * (x + y))
- : unit = ()

```

考虑一下，怎么使用类C语言干净地表达上述逻辑？C++程序员可能会想到利用函数重载或者模板类型特化的方式。重载某种意义上也是一种模式匹配，只不过C++的语法没那么优雅而已。

较新的高级语言几乎都是**多范式的**，它们的主体是命令式的（以便于大众接受），但又纷纷从函数式语言中吸收这些特性，来实现强大的表达能力。例如，本人很喜欢的Rust，支持类型推导、参数多态、模式匹配，其“枚举类型”enum就支持代数数据类型，而且Rust而且还支持检查是否穷尽了那个ADT所有可能的类型：

```
enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}

fn main() {
    let msg = Message::ChangeColor(0, 160, 255);
    match msg {
        Message::Quit => println!("The Quit variant has no data to d
estructure."),
        Message::Move { x, y } => println!("Move in the x direction
{} and in the y direction {}", x, y);
        Message::Write(text) => println!("Text message: {}", text),
        Message::ChangeColor(r, g, b) => println!("Change the color
to red {}, green {}, and blue {}", r, g, b)
    }
}
```

Miranda / Haskell 的惰性计算和纯函数

David Turner开发了Miranda语言，这是一个实现了 **惰性计算(Lazy evaluation)** 的 **纯函数(Pure function)** 编程语言。Haskell继承了Miranda所发展出的概念，并发展至今。

惰性计算

λ 演算可以用 β -规约来做计算，但是，一个表达式可以有多种做被 β -规约的方式，例如，表达式 $(\lambda x. x + x) (5 * 4)$ 可以有不同的规约方式

1. $(\lambda x. x + x) (5 * 4) \Rightarrow (5 * 4) + (5 * 4)$
 这是最左优先的**normal-order reduction**，对函数应用"f e"，它总是先将"f e"先规约，然后再将e规约成值。也就是说，参数总是在它被使用时被规约。因此，常常也被称为**传名调用(call-by-name)**
2. $(\lambda x. x + x) (5 * 4) \Rightarrow (\lambda x. x + x) 20$
 这是所谓的**applicative-order reduction**，对于函数应用"f e"，它先将e规约成值v，然后规约"f v"。也就是说，参数总是在调用之前被规约。因此，常常也被称为**传值调用(call-by-value)**

传名调用的好处是，如果表达式存在一个不能被进一步规约的形式，那么总是可以在有限步内找到它。对于传值调用，这个命题不成立。例如，表达式 $(\lambda x. 1)$

e ，假定 e 是无限递归的表达式，传名调用仍然可以将它规约成1，但是传值调用却会陷入无限循环。

但是，传名调用有一个致命缺陷，先规约“f e”再规约e的值可能导致e被反复计算，造成严重的性能问题。在前面的例子中， $5*4$ 用传名调用被算了2次。可以想象，当表达式进行多层嵌套时会产生什么样的后果。

```
-- Normal-order reduction :
      (λ x. x + x) (5 * 4)
=> (5 * 4) + (5 * 4)
=> 20 + (5 * 4)
=> 20 + 20
=> 40
-- Applicative-order reduction :
      (λ x. x + x) (5 * 4)
=> (λ x. x + x) 20
=> 20 + 20
=> 40
```

为了性能以及实现方便，Lisp/ML等语言均使用传值调用，但是Miranda走出了一条与众不同的路：惰性计算。

使用惰性计算，我们可以这么做：

当规约 $(\lambda x. e_1) e_2$ 时，我们先创建一个指向 e_2 的指针。然后，将 $(\lambda x. e_1) e_2$ 规约，将 e_1 中所有的 x 用指向 e_2 的指针替代，得到 e_1' 。接着，如果我们在规约 e_1' 的过程中需要规约那个指针，我们就规约它指向的表达式 e_2 。

重点来了：这只需要规约一次，下一次我们再规约 e_1 时，我们不需要再规约 e_2 了，因为它已经被规约过了（记忆化）。更进一步地，如果 e_1' 不包含那个指针，我们根本不需要规约 e_2 （前面那个不会终止的 e 就这样被解决了）。

从这个意义上来说，我们又将惰性计算称作**按需调用(call-by-need)**，我们只需要 e_2 时计算它，而且它只会被计算1次。

惰性计算打开了新世界的大门：

1. 将**控制流**(Control flow,包括了我們熟知的无条件跳转、条件语句、循环语句、异常捕获等等)定义为**抽象**而不是**原语**。按我的理解，控制流不必以关键

字的形式直接受编译器支持，而可以通过库的形式，用更基础的功能定义出来，这使得用户可以定义自己的控制流。

还记得我前面举例的C# Linq吗？Linq的一大特征就是延迟计算，它通过C#的ExpressionTree实现，而且Linq是个库（虽然为了使用方便，也加入了上下文关键字，但是会被编译器转换成链式函数调用）。

2. 定义无限数据结构。例如，按顺序返回自然数的迭代器（自然数是无穷多的）。

这里举例的这种迭代器通常也被称为Generator，在各种主流语言以yield的形式出现。由此出发，还可以发明出协程(Coroutine)的相关原语，这是现代语言中一大异步方案，如今最著名的例子应当是Golang. 此外，Coroutines提案现已进入C++20标准。

3. 避免不必要的计算，提升性能，并且提前查出复合表达式的错误。

惰性计算当然也存在一些问题。它的实现上有一些开销，而且很难分析它的性能表现，因为它的求值顺序和时机不能确定。Haskell默认使用惰性计算，是好是坏众说纷纭。

纯函数

1985年，Miranda发行后，惰性计算的函数式编程语言受到了很多人的关注。到1987年，已经出现了十多种类似语言，其中Miranda使用最为广泛。但是Miranda是商业产品，不属于公共领域，在1987年的函数式编程语言与计算机结构大会(FPCA '87)上，参会者一致同意形成一个委员会来为这样的语言定义一种开放性的标准，并于1990年定义了Haskell编程语言。

Haskell可以说是Miranda标准化的产物，它是一门支持惰性求值、模式匹配、列表解析、类型类(Typeclasses)、类型多态的**纯函数**编程语言。Haskell的类型系统有着完善的数学理论支持，非常强大，在学术界有着大量的研究者，其不少功能的背后都是一篇又一篇的paper，但也非常难学。

前面我们提到了，Haskell的另一大特色是**纯函数**。纯函数的特点是无副作用，这有很多好处：

1. 可重复、无状态。这不仅便于测试，而且使得延迟计算真正成为可能——因为它的重复计算结果一定是相同的，所以只要算一遍。而非纯函数由于可以改变环境上下文（副作用），所以不能保证下次的结果仍然相同。
2. 不可变性。这允许编译器做比较激进的优化（GHC的Inliner质量很高），因为不可变性大大限制了程序的局部行为。而且这免除了数据竞争，因为函数的结果与调用方的计算顺序无关。这在并行计算方面有着得天独厚的优势。现在是2020年，在单核性能停滞不前的今天，并行计算有多重要不言而喻。

但是，纯函数也是有缺陷的：

1. 副作用在真实世界是存在的。像I/O这样的操作，本质上是一定有副作用的。这样的问题强行用纯函数建模，会将问题复杂化（可参考Haskell的Monad的复杂程度）。
2. 无状态可能导致一些算法的复杂度上升，便于并行带来的优势可能被算法复杂度的差别所抵消（更何况，有些问题天生是串行的）。

Haskell的入门教程可阅读[Real World Haskell](#)。下面的例子计算了斐波那契数列。GHC会进行尾递归优化，使得下列代码实际上是O(n)的。

```
{-# LANGUAGE BangPatterns #-}
fib n = go n (0,1)
  where
    go !n (!a, !b) | n==0      = a
                  | otherwise = go (n-1) (b, a+b)
```

Haskell轶事

Haskell中有一个叫做Monad的概念，出名地难懂，涉及到了数学上的范畴论等概念，以至于一句开玩笑的话流传甚是广泛（原文是用来黑Haskell的，但结果以讹传讹，吓跑了无数人）：

*A monad is just a monoid in the category of endofunctors,
what's the problem?*

一个单子不过就是自函子范畴上的一个么半群而已，有什么问题吗？

这句话出自 [A Brief, Incomplete, and Mostly Wrong History of Programming Languages](#)

为什么我们需要FP?

开个玩笑

你会和钱过不去吗？金融界用FP的可多了：

Haskell渣打在用，Scala摩根斯坦利在用，OCaml Jane Street在用，Clojure花旗在用

编程语言的一个重要发展节点是“结构化编程”的提出，而“结构化”和“非结构化”的一个重要差别是模块化设计，这带来了生产力的大幅提高。首先，小的模块方便编写；其次，通用的模块可以被重用，使得后续的软件迭代更加迅速；再者，模块可以被独立地测试，减少调试所需要消耗的时间。

但是，模块化编程需要将问题划分为子问题，并且解决这些子问题，并将它们**组合**在一起。因此，我们需要能将“子问题”组合的“胶水”。函数式编程就是这样的胶水，它提供了非常重要的新思路，其中的重点是：

1. 将函数组合：高阶函数使得函数的可组合性更上一层楼。
2. 将程序组合：惰性计算带来了模块化设计的新形式。更确切地说，可以参考Why Functional Programming Matters的观点：
 - Modular programming needs powerful glue
 - Lazy evaluation enables new forms of modularity; in particular, **separating generation from selection.**
 - Non-strict semantics means that unrestricted beta substitution is OK

函数式编程非常强调“组合性”，**编程语言的很大一部分研究，就是研究如何提高可组合性。**

上世纪80-90年代的时候，谈面向对象编程(OOP)是一种政治正确，现在的函数式编程则成了程序员口中的时尚。但是，必须清醒地认识到，软件开发**没有银弹**，不存在能够一劳永逸解决所有问题的完美方案。

命令式编程在上世纪之所以能占据工业界主流，和**性能**问题紧密相关，因为命令式的表达更容易映射到机器的实现（条件判断、循环等结构可以同汇编直接对应）。而且，某些函数式编程语言的概念确实难懂。

而在2020年的现在，程序员的生产力显然更重要，而且，不要小看编译器，现在的编译器已经发展到了一些FP特性都能零开销抽象的水平。下面是一段音频解码器代码（出自 *The Rust Programming Language*），典型的有高性能需求的场景。这里创建了一个迭代器，使用了两个适配器，接着消费了它的值。看起来很函数式，这有代价吗？实际上，它会被编译成**与手写汇编相同的代码**。遍历 `coefficients` 完全用不到循环，编译器明白这里会迭代12次，做了循环展开，而且所有系数都存储在了寄存器中。不仅如此，迭代器的特性使得运行时的数组边界检查是没必要的，所以编译器当然不会生成边界检查。

```
let buffer: &mut [i32];
let coefficients: [i64; 12];
let qlp_shift: i16;

for i in 12..buffer.len() {
    let prediction = coefficients.iter()
                                .zip(&buffer[i - 12..i])
                                .map(|(&c, &s)| c * s as i64)
                                .sum::<i64>() >> qlp_shift;

    let delta = buffer[i];
    buffer[i] = prediction as i32 + delta;
}
```

现代的主流语言几乎都是多范式的，而且纷纷加入FP特性，这里的例子数不胜数：

1. 2010年以后，几门影响力较大的新语言，像Rust, Kotlin, Swift，都是在类C风格之下，加入了大量的函数式编程特性。
2. C++也在C++11标准中加入了Lambda表达式的支持，大大提升了 `<algorithm>` 的可用性。即将到来的C++20，其重磅功能Ranges更是充满着一股FP的味道。
3. 就连老古板Java，都在Java 8中加入了Lambda表达式和Stream API。
4. 当今异步计算盛行，各种语言纷纷加入语法或库来支持这种写法。其中很流行的一种方案 `async/await` 出自C#，Javascript / Python / Rust纷纷都采用了这个方案。而C#的这个功能出自函数式编程语言F#的 `Computation expression`。
5. 当今的Web前端三大框架之一的React，也在力推函数式组件，以提高组合性，同时便于实现声明式的UI模型。

这充分说明，函数式编程对表达能力的提升有着巨大的帮助。

Q&A

- 问：PPT6页的JS代码是多参数lambda表达式的实现吗？看上去有点奇怪，请问参数y是怎么传到内部函数中的呢？

答：（首先修正一个错误，我的PPT贴图在function(y)前面漏了一个return）

这是所谓的“闭包”概念，就是是能够捕获所处环境中的变量的函数。内层的匿名函数捕获（也就是绑定）了其所处环境的变量y，所以就可以引用了。

如果你想要实验的话，可以这么试试：

在浏览器中，打开开发者工具的控制台，输入：

```
var curried = function(x) {return function(y) { return x; } };
```

回车，然后输入

```
curried(1);
```

你应该可以看到类似这样的返回值

```
f (y) { return x; }
```

再输入

```
curried(1)(2);
```

你应该可以看到返回了 1

相关概念：Closure, Currying, Partial Application

- 问：组合与递归有何区别？

答：这是两个完全不同的概念。

组合就是组合的字面意义：给你两个东西，怎么产生一个更大的东西。

递归则是一种对问题的分解，是一种通过重复将问题分解为同类的子问题而解决问题的方法。

在计算机科学中，函数的组合就是将简单的函数组合成复杂的函数的过程。几乎所有的程序语言都允许函数组合，但是在函数被当作一等公民的语言中，这样的操作更为容易。类似地，对象组合就是将简单的对象组合成复杂的对象的过程。

例如，C语言代码

```
y = g(x);  
z = f(y);
```

两个函数调用可以被组合成一个： $z = f(g(x))$;

此外，我们可以说，两个语句被组合成了一个顺序执行的程序，因为从上到下排列的语句能表达出“然后执行这个操作”的意思，对于顺序执行是可组合的。

但是，如果中间出现了goto，它就破坏了这个逻辑，你没法将上面这部分语句和下面这部分语句组合起来，变成保持顺序执行语义的更大程序。这也是为什么我们要远离goto

```
y = g(x);  
/* code */  
goto label;  
/* code */  
z = f(y);
```

再举一个例子，在Golang的try()错误处理提案中(<https://github.com/golang/proposal/blob/master/design/32437-try-builtin.md>)，因为try“函数”会改变控制流，这导致这样的写法写不出来了

```
h = try(f(try(g(x))))
```

而必须这么写

```
t := try(g(x))  
h = try(f(t))
```

这个“函数”就是不可组合的。题外话，这个提案最终被毙掉了。

参考资料

本文的内容离不开下列资料的支持：

1. Lambda演算和组合子逻辑的历史内容主体总结、翻译自[Cardone, Felice & Hindley, J.. \(2006\). History of lambda-calculus and combinatory logic.](#)，这篇文章启发我查阅了很多相关资料。

2. Lambda演算的相关资料参考了下列内容：

[Lambda calculus - Wikipedia](#)

[Programming Languages and Lambda Calculi](#)

[Lambda演算学习笔记](#)

[λ演算\(Lambda Calculus\)入门基础（一）：定义与归约](#)

[λ演算\(Lambda Calculus\)入门基础（二）：丘奇编码\(Church Encoding\)](#)

3. 函数式编程的历史内容主体总结、翻译自：

[Gang Tan, Penn State \(2004\): A Brief History of Functional Programming](#)

4. 函数式编程意义的阐述，参考了著名论文：

[J. Hughes, Why Functional Programming Matters, The Computer Journal, Volume 32, Issue 2, 1989, Pages 98–107](#)

5. 我查阅的资料来源还包括以下网站，前面大都给出了链接来源：各个语言的英文维基百科和中文维基百科，包括Lisp, Algol, ML, Haskell等等；知乎，主要是一些奇怪的问题、想法、八卦，以及一些资料翻译；语言教程，如OCaml的官方Tutorial，Haskell书籍Real World Haskell等。另外，类型系统的介绍可参考[Type systems](#)